Artifical Intelligence

BS (CS) Spring 2026

TF-IDF Manual

Chapter 1 — What is TF-IDF?

1.1 The Problem TF-IDF Solves

Computers cannot read text the way humans do. To apply machine learning, we must convert words into numbers. The simplest approach is counting — how many times does each word appear in a document? But this approach has a serious flaw.

The Problem with Raw Word Counts

Imagine two documents about football:

Document 1: "The player scored a goal. The referee stopped the game."

Document 2: "The goal changed the match. The team celebrated the goal."

The word "the" appears 3 times in each. But "the" tells us nothing about what the document is about.

The word "goal" appears 2 times in Document 2 — and that IS informative.

Raw counts reward common words (the, a, is, was) and punish rare-but-important words.

TF-IDF solves this by combining two measurements:

- **TF (Term Frequency)** — how often a word appears in THIS document.
- **IDF (Inverse Document Frequency)** — how rare the word is ACROSS ALL documents.

A word scores high in TF-IDF when it appears frequently in one document but rarely in the overall collection. That pattern signals that the word is important to that specific document.

1.2 Real-World Intuition

Intuition: Library Card Catalogue

Think of TF-IDF like a librarian ranking books by relevance to your search query.

If you search for "photosynthesis":

- The word "the" appears in every book → IDF is near zero → useless for ranking.
- The word "photosynthesis" appears in very few books → IDF is high.
- A biology textbook uses "photosynthesis" 40 times → TF is high.

→ That biology textbook gets a high TF-IDF score → top of the results.

TF-IDF is the mathematical version of this librarian intuition.

Chapter 2 — The Mathematics

2.1 Term Frequency (TF)

Term Frequency measures how often a term t appears in document d, normalized by the total number of terms in d so that longer documents are not unfairly penalized.

```
TF(t, d)  =  (Number of times term t appears in document d)
             ____________________________________________________
                          (Total number of terms in document d)
```

TF Example

Document d = "the cat sat on the mat the cat"
Total terms in d = 8

TF("the", d) = 3 / 8 = 0.375
TF("cat", d) = 2 / 8 = 0.250
TF("sat", d) = 1 / 8 = 0.125
TF("mat", d) = 1 / 8 = 0.125

2.2 Inverse Document Frequency (IDF)

IDF measures how rare a term is across the entire document collection. If a term appears in nearly every document, it is not useful for distinguishing documents. The logarithm compresses the scale so very rare terms are not given infinite weight.

```
IDF(t, D)  =  log( N / df(t) )

N     = total number of documents in the collection
df(t) = number of documents that contain term t
log   = natural log (ln) or log base 10 (both are used)
```

A smoothed version (used by scikit-learn) adds 1 to avoid division by zero:

```
IDF(t, D)  =  log( (N + 1) / (df(t) + 1) )  +  1

The +1 inside log avoids zero for terms in every
                    document.
The +1 outside ensures IDF is never zero.
```

IDF Example — 5 Documents

Collection D = 5 documents (N = 5)

"the" appears in all 5 documents → df=5 → IDF = log(5/5) = log(1) = 0
"cat" appears in 2 documents → df=2 → IDF = log(5/2) = log(2.5) ≈ 0.916
"sat" appears in 1 document → df=1 → IDF = log(5/1) = log(5) ≈ 1.609

"the" has IDF = 0, so it will contribute nothing to TF-IDF scores.
"sat" has IDF = 1.609 — it is a distinctive term.

2.3 The TF-IDF Score

The final TF-IDF score is simply the product of TF and IDF:

```
TF-IDF(t, d, D)  =  TF(t, d)  ×  IDF(t, D)
```

A high TF-IDF score means: the term appears frequently in this document AND rarely in the rest of the collection. That is the definition of an important, document-specific term.

2.4 Full Worked Example — Step by Step

Let us compute TF-IDF manually for a tiny collection of 3 RACE-like passages.

Step 1 — The Collection

Doc	Text
D1	The student studied hard and passed the exam successfully.
D2	The teacher graded the exam papers carefully.
D3	The student learned new concepts and understood the lesson.

Step 2 — Compute TF for 'student' in D1

After removing stopwords (the, and), D1 becomes: [student, studied, hard, passed, exam, successfully]

```
TF('student', D1) = 1 / 6 = 0.167
```

Step 3 — Compute IDF for 'student'

'student' appears in D1 and D3. So df('student') = 2, and N = 3.

```
IDF('student') = log(3 / 2) = log(1.5) ≈ 0.405
```

Step 4 — Compute TF-IDF

```
TF-IDF('student', D1) = 0.167 × 0.405 ≈ 0.068
```

Step 5 — Compare All Key Terms in D1

Term	TF in D1	IDF	TF-IDF
the	0.000*	0.000	**0.000 ← useless stopword**
student	0.167	0.405	**0.068**
studied	0.167	1.099	**0.183 ← high (unique to D1)**
exam	0.167	0.405	**0.068 (also in D2)**
successfully	0.167	1.099	**0.183 ← high (unique to D1)**

* 'the' was removed as a stopword before vectorization. In practice scikit-learn's TfidfVectorizer does this automatically with stop_words='english'.

Key Insight from Step 5

"studied" and "successfully" get the highest TF-IDF scores in D1.
Why? They appear in D1 but in NO other document → IDF is high (log(3/1) = 1.099).
"exam" has a lower score because it also appears in D2 → less distinctive.
"the" has a score of 0 → it tells us nothing about D1 specifically.

Chapter 3 — TF-IDF Vectors & Cosine Similarity

3.1 Documents as Vectors

Once TF-IDF scores are computed for every term in every document, each document becomes a vector — a list of numbers, one per unique term in the vocabulary. Most values are zero (the term is absent).

Example — Vocabulary and Vectors

Vocabulary (after stopword removal): [cat, dog, exam, mat, sat, student, studied]

Doc	cat	dog	exam	mat	sat	student	studied
D1	0	0	0.07	0	0	0.07	0.18
D2	0	0	0.07	0	0	0	0
D3	0	0	0	0	0	0.07	0

3.2 Cosine Similarity

Once documents are vectors, we can measure their similarity. Cosine similarity measures the angle between two vectors. If two document vectors point in the same direction, their cosine similarity is 1 (identical). If they are unrelated, the similarity approaches 0.

```
cosine_similarity(A, B)  =  (A · B)  /  (||A|| × ||B||)

A · B   = dot product = sum of (A_i × B_i) for each
                        dimension i
||A||   = Euclidean length of vector A = sqrt(sum of
                        A_i²)
```

Why Cosine, Not Euclidean Distance?

Euclidean distance is affected by document length — a long document and a short document about the same topic would appear far apart just because of length. Cosine similarity is length-independent: it measures direction, not magnitude. This makes it ideal for text.

Cosine Similarity in the RACE Project

In your project you will use cosine similarity to:

- Rank which passage sentence best matches a question (Model A — verification feature)
- Find medium-similarity sentences for distractor candidates (Model B)
- Score hint sentences for relevance to the correct answer (Model B — hint generator)

scikit-learn makes this one line of code:

```
from sklearn.metrics.pairwise import cosine_similarity
sim_matrix = cosine_similarity(tfidf_matrix_A, tfidf_matrix_B)
```

Chapter 4 — TF-IDF with scikit-learn

4.1 TfidfVectorizer — Core Parameters

scikit-learn's TfidfVectorizer handles tokenization, stopword removal, TF-IDF computation, and L2-normalization in one object. Understanding its parameters is essential.

Parameter	Default	What It Does
max_features	None	Keep only the top N terms by frequency. Use 5000–20000 for RACE to reduce memory.
stop_words	None	Set to 'english' to remove stopwords (the, a, is...). Strongly recommended.
ngram_range	(1, 1)	Unigrams only. Set (1,2) to include bigrams (e.g. 'reading comprehension').
min_df	1	Ignore terms that appear in fewer than min_df documents. Use 2–5 to remove typos.
max_df	1.0	Ignore terms appearing in more than max_df fraction of docs. Use 0.9 to remove near-stopwords.
sublinear_tf	False	Apply log(1+TF) instead of raw TF. Reduces effect of very frequent terms. Recommended: True.
norm	'l2'	Normalize each document vector to unit length. Required for cosine similarity to work correctly.
analyzer	'word'	Tokenize by word. Can set to 'char_wb' for character n-grams.
token_pattern	r'(?u)\b\w\w+\b'	Regex for tokenization. Default matches words of 2+ characters.

4.2 Basic Usage

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

# Load RACE data from Kaggle
train_df = pd.read_csv('data/raw/train.csv')
```

```
# Use article text as the corpus
corpus = train_df['article'].tolist()

# Create and fit the vectorizer
vectorizer = TfidfVectorizer(
    max_features = 10000,    # Vocabulary size
    stop_words   = 'english',# Remove common English stopwords
    sublinear_tf = True,     # Use log(1+TF) to dampen high frequencies
    ngram_range  = (1, 2),   # Include unigrams and bigrams
    min_df       = 2,        # Ignore very rare terms
    max_df       = 0.95,     # Ignore near-universal terms
)

# Fit on training data and transform
X_train = vectorizer.fit_transform(corpus)

print('Matrix shape:', X_train.shape)
# Output: (87866, 10000)  ← 87866 documents, 10000 features

print('Vocabulary sample:', list(vectorizer.vocabulary_.items())[:5])
```

4.3 Inspecting the Vocabulary

```
import numpy as np

# Get feature names (terms in vocabulary)
feature_names = vectorizer.get_feature_names_out()

# Find TF-IDF scores for a single document
doc_idx = 0   # First article in training set
doc_vector = X_train[doc_idx]

# Get non-zero entries sorted by TF-IDF score
scores = zip(feature_names, np.asarray(doc_vector.todense()).flatten())
sorted_scores = sorted(scores, key=lambda x: x[1], reverse=True)

print('Top 10 TF-IDF terms for document 0:')
for term, score in sorted_scores[:10]:
    print(f'  {term:30s}  {score:.4f}')
```

4.4 Transforming New Data (Test Set)

CRITICAL: only call fit_transform() on training data. Use transform() on validation and test data. Calling fit_transform() on test data would cause data leakage — the vectorizer would learn from test set statistics.

```
val_df  = pd.read_csv('data/raw/val.csv')
test_df = pd.read_csv('data/raw/test.csv')
```

```
# CORRECT: transform only (vectorizer already fit on train)
X_val  = vectorizer.transform(val_df['article'].tolist())
X_test = vectorizer.transform(test_df['article'].tolist())

# WRONG — never do this on val/test:
# X_val = vectorizer.fit_transform(val_df['article'].tolist())  # DATA
LEAKAGE!
```

⚠ Data Leakage Warning

If you fit the TfidfVectorizer on test data, your evaluation metrics will be artificially inflated because the model has seen the test distribution.

Rule: fit_transform() → training set ONLY.
transform() → validation and test sets.

Save the fitted vectorizer with joblib so you can reload it for inference.

Chapter 5 — Applying TF-IDF to the RACE Project

5.1 Overview of Use Cases

Use Case	Model	How TF-IDF Is Used
Answer verification	**Model A**	Vectorize (article + question + option). Cosine similarity + TF-IDF scores become features for LR/SVM classifier.
Question generation	**Model A**	Score each sentence in article vs. correct answer by cosine similarity. Top sentence = question stem candidate.
Distractor candidates	**Model B**	Retrieve medium-similarity sentences from article as distractor source material.
Hint ranking	**Model B**	Rank article sentences by cosine similarity to question. Top = Hint 3, bottom = Hint 1.
Word overlap feature	**Model A**	Jaccard similarity between TF-IDF top-K terms of question and each option.

Chapter 6 — Saving, Loading & Best Practices

6.1 Saving the Fitted Vectorizer

The fitted TfidfVectorizer must be saved alongside the trained classifier. Without it, you cannot vectorize new input text at inference time.

```
import joblib

# After fitting:
joblib.dump(vectorizer, 'models/model_a/tfidf_vectorizer.pkl')
joblib.dump(classifier, 'models/model_a/svm_classifier.pkl')

# At inference time:
vectorizer  = joblib.load('models/model_a/tfidf_vectorizer.pkl')
classifier  = joblib.load('models/model_a/svm_classifier.pkl')

def predict_answer(article, question, options):
    best_opt, best_score = None, -1
    for label, text in options.items():
        combined = f'{article} {article} {question} {text}'
        X = vectorizer.transform([combined])
        prob = classifier.predict_proba(X)[0][1]  # prob of correct
        if prob > best_score:
            best_score, best_opt = prob, label
    return best_opt, best_score
```

6.2 Common Mistakes & Fixes

Mistake	Fix
Calling fit_transform() on test/val data	Only call fit_transform() on training data. Use transform() on val and test.
Not saving the vectorizer with the model	Always save vectorizer with joblib. A model without its vectorizer is useless.
Using max_features=None on RACE (memory crash)	Set max_features=10000–20000. Full RACE vocabulary has 100K+ terms.
Not setting sublinear_tf=True	With sublinear_tf=True, TF = log(1+count). Prevents very frequent terms from dominating.
Using default stop_words=None	Set stop_words='english'. Stopwords inflate feature dimensions without adding information.
Concatenating article + option in wrong order	Give more weight to article: repeat it. Combined = article + article + question + option.

Mistake	Fix
Using dense matrix (toarray()) for large RACE data	Keep as sparse matrix (scipy.sparse). Converting to dense causes memory errors on RACE.

6.3 Hyperparameter Tuning for TfidfVectorizer

Use GridSearchCV to find the best TF-IDF settings together with your classifier. Wrap both in a Pipeline so the vectorizer is treated as part of the model.

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words='english')),
    ('clf',   LogisticRegression(max_iter=1000)),
])

param_grid = {
    'tfidf__max_features': [5000, 10000, 15000],
    'tfidf__ngram_range':  [(1,1), (1,2)],
    'tfidf__sublinear_tf': [True, False],
    'clf__C':              [0.1, 1.0, 10.0],
}

grid_search = GridSearchCV(
    pipeline,
    param_grid,
    cv=3,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=2,
)

grid_search.fit(train_texts, y_train)
print('Best params:', grid_search.best_params_)
print('Best F1:',     grid_search.best_score_)
```

Chapter 7 — Strengths, Limitations & Alternatives

7.1 Strengths of TF-IDF

- Simple to understand and implement — the math is transparent and auditable.
- Fast to compute — vectorizing 87,000 RACE articles takes seconds on a laptop.
- Memory-efficient — TF-IDF matrices are sparse (most values are 0).
- No training required — TF-IDF is a deterministic transformation, not a learned model.
- Works well with linear classifiers (LR, SVM) which are fast and interpretable.
- Interpretable features — you can inspect which terms drove a prediction.

7.2 Limitations of TF-IDF

- No semantic understanding — 'student' and 'pupil' are treated as completely different terms even though they mean the same thing.
- No word order — 'dog bites man' and 'man bites dog' produce identical TF-IDF vectors.
- Out-of-vocabulary problem — words not seen during fit() get a score of 0, even if they are meaningful.
- Sparsity — most entries are zero. High-dimensional sparse vectors can be hard to work with for some models.
- Poor at paraphrase detection — two sentences with the same meaning but different words will have low cosine similarity.

Why This Is Fine for Your 2-Week Project

These limitations are real but manageable within your project scope:

- Semantic gap → partially addressed by adding Word2Vec features in Model B.
- Word order ignored → your classifier learns from the label distribution, not syntax.
- OOV words → RACE is a fixed dataset; test vocabulary is similar to train vocabulary.

Neural embeddings (BERT, Word2Vec) solve these problems but require more time and compute.
TF-IDF is the right tool for a 2-week classical-ML project.

7.3 TF-IDF vs. Other Representations

Method	Captures Semantics	Speed	Use in Your Project
Bag of Words	**No**	Very Fast	Baseline only — use TF-IDF instead
TF-IDF	**No**	Very Fast	Primary method for all features
Word2Vec (avg)	**Yes**	Fast	Model B distractor generation
GloVe (avg)	**Yes**	Fast	Alternative to Word2Vec
BERT embeddings	**Yes**	Slow	Out of scope — not allowed

Chapter 8 — Practice Exercises

Complete these exercises in order. Each builds on the previous. Submit your notebook with all outputs visible.

Exercise 1 — Manual TF-IDF Calculation

Given the following 3 sentences from a RACE article:

S1: "The ancient Silk Road connected China to Europe."
S2: "Merchants traded silk, spices, and glass along the route."
S3: "The route passed through deserts and mountains."

Tasks:

a) Remove stopwords. List the remaining tokens for each sentence.
b) Compute TF for the term 'route' in S2 and S3.
c) Compute IDF for 'route' across the 3 sentences.
d) Compute TF-IDF('route', S2) and TF-IDF('route', S3).
e) Which sentence has a higher TF-IDF score for 'route'? Why?

Exercise 2 — scikit-learn Vectorization on RACE

Load the RACE training set from Kaggle (train.csv).

a) Fit a TfidfVectorizer on the 'article' column. Use:
max_features=10000, stop_words='english', sublinear_tf=True
b) Print the shape of the resulting matrix.
c) Find the 10 terms with the highest average TF-IDF score across all articles.
d) For article index 100, print the top 5 terms by TF-IDF score.
e) Transform the val.csv articles using the same vectorizer (do NOT refit).

Exercise 3 — Cosine Similarity

Using the vectorizer from Exercise 2:

a) Pick any 3 articles from the test set.
b) Compute the 3×3 cosine similarity matrix between them.
c) Which two articles are most similar? Read both and explain why.
d) For article index 0, rank ALL sentences in the article by cosine similarity to the corresponding question. Print the top 3 sentences.
e) Does the top-ranked sentence contain the correct answer? Report your hit rate

across 50 random samples.

Exercise 4 — Build the Full Feature Pipeline

Implement build_verification_features() from Chapter 5.

a) Build X_train and y_train for the first 10,000 rows of train.csv.
b) Verify class balance: y_train.mean() should be close to 0.25.
c) Add the 6 cosine similarity features from Section 5.3.
d) Train a Logistic Regression on the combined features.
e) Evaluate on val.csv. Report Accuracy, Macro F1, and Confusion Matrix.
f) Which feature (TF-IDF or cosine similarity) is more important?
Use clf.coef_ or permutation_importance to investigate.

Exercise 5 — Distractor Generation

Implement get_distractor_candidates() from Chapter 5.

a) Run it on 20 random RACE test samples.
b) For each sample, print: the correct answer + 3 generated distractors.
c) Compute BLEU-1 score between your generated distractors and the original dataset options (B, C, D when A is correct, etc.).
d) Compute pairwise cosine distance between your 3 distractors. Higher = more diverse. Report the average across 20 samples.
e) Manually rate 5 samples for plausibility (1–5 scale). Discuss.

— End of TF-IDF Manual —